BCD II: Business Components Development II

Comprehensive Lecture Notes on J2EE Architecture

Prepared by: K A A C Lakshan

Senior Lecturer

Java Institute for Advanced Technology (JIAT)

Table of Contents

Contents

1. Transaction Management in EJB

Transaction management is one of the most crucial elements in developing robust enterprise applications. In the realm of Java EE and Business Component Development (BCD), handling transactions ensures that complex operations, which may span multiple databases or message queues, maintain data integrity.

A transaction is a unit of work that must be completed fully or not at all. This is governed by the ACID properties: Atomicity, Consistency, Isolation, and Durability.

1.1 The ACID Properties

• Atomicity: Ensures that all operations within the work unit are completed successfully. If not, the transaction is aborted at the point of failure and all previous operations are rolled back to their former state.
• Consistency: Ensures that the database properly changes states upon a successfully committed transaction.
• Isolation: Enables transactions to operate independently of and transparent to each other.
• Durability: Ensures that the result or effect of a committed transaction persists in case of a system failure.

1.2 Container-Managed Transactions (CMT)

In Container-Managed Transactions, the EJB container handles the transaction boundaries. The developer does not write code to start, commit, or roll back transactions. Instead, they use annotations to instruct the container on how to manage them. This is the preferred method in EJB due to its simplicity and separation of concerns.

```
@Stateless
@TransactionManagement(TransactionManagementType.CONTAINER)
public class OrderServiceBean implements OrderService {

    @PersistenceContext
    private EntityManager em;
```

```
    @TransactionAttribute(TransactionAttributeType.REQUIRED)
    public void placeOrder(Order order) {
        em.persist(order);
        // Container commits transaction automatically upon
successful return
    }
}
```

1.3 Transaction Attributes in CMT

The EJB container provides six transaction attributes that dictate how business methods participate in transactions:

1. REQUIRED: If a client is running within a transaction and invokes the enterprise bean's method, the method executes within the client's transaction. If the client is not associated with a transaction, the container starts a new transaction before running the method.

2. REQUIRES_NEW: The container always starts a new transaction. If the client invokes the enterprise bean's method while in a transaction, the container suspends the client's transaction, starts a new transaction, executes the method, and resumes the client's transaction after the method completes.

3. MANDATORY: The method must execute within a transaction. If the client is not associated with a transaction, the container throws a TransactionRequiredException.

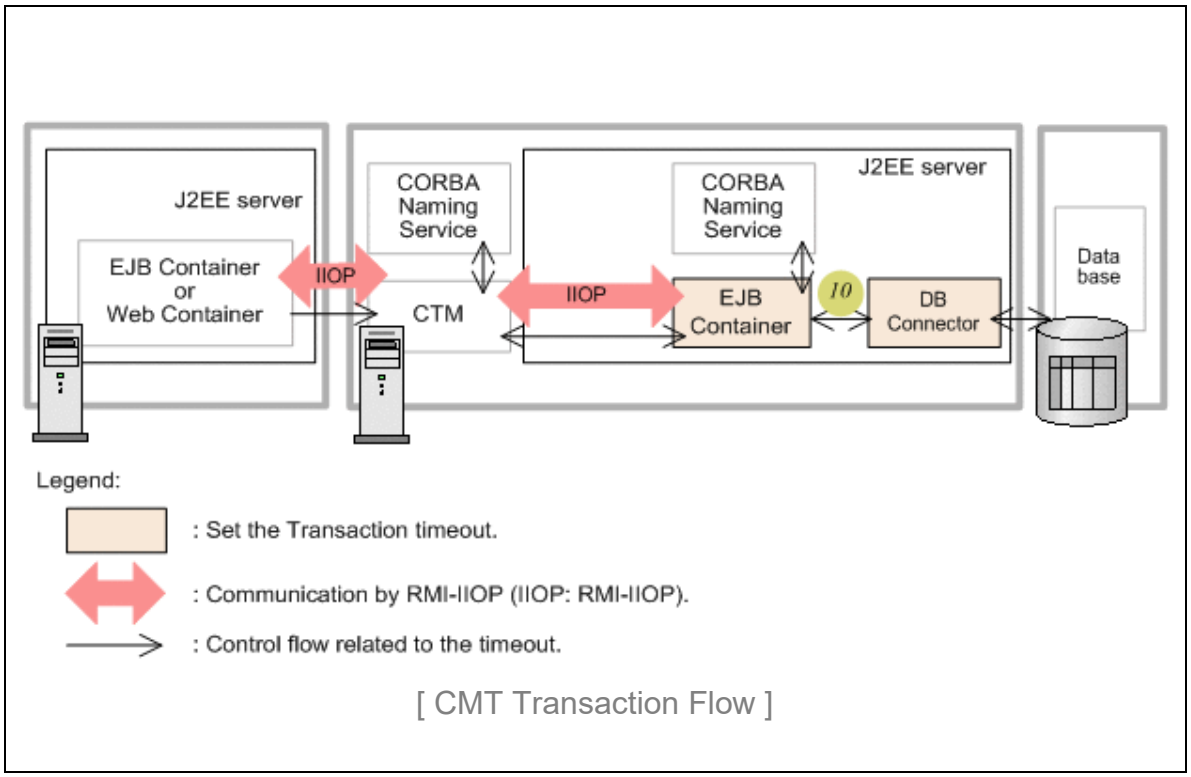
4. SUPPORTS: The method does not require a transaction, but it will execute within one if the client provides it.

5. NOT_SUPPORTED: The container suspends the client's transaction before invoking the method and resumes it afterwards.

6. NEVER: The method must never execute within a transaction. Throws an exception if a transaction is present.

Diagram: CMT Transaction Flow

Description: A flowchart showing the decision tree of a client calling an EJB method. The flow splits based on the TransactionAttribute (REQUIRED, REQUIRES_NEW, etc.) and whether the client already holds a transaction, illustrating suspension and creation of transaction contexts.



[CMT Transaction Flow]

1.4 Bean-Managed Transactions (BMT)

With Bean-Managed Transactions, the developer is responsible for writing the code that marks the boundaries of a transaction using the UserTransaction interface. This provides finer control, such as keeping a transaction open across multiple method calls within a stateful session bean, but increases code complexity.

```
@Stateless
@TransactionManagement(TransactionManagementType.BEAN)
public class PaymentServiceBean {

    @Resource
    private UserTransaction utx;
```

```
    @PersistenceContext
    private EntityManager em;

    public void processPayment(Payment payment) {
        try {
            utx.begin();
            em.persist(payment);
            // Additional business logic
            utx.commit();
        } catch (Exception e) {
            try {
                utx.rollback();
            } catch (SystemException se) {
                se.printStackTrace();
            }
        }
    }
}
```

1.5 Best Practices & Pitfalls

• Always prefer CMT over BMT unless you have a specific use case requiring fine-grained programmatic control.

• Be careful with REQUIRES_NEW in loops, as it can exhaust connection pools.

• Do not swallow exceptions in CMT; if an exception is caught and not rethrown, the container might not realize the transaction needs to be rolled back unless explicit rollback methods are called.

2. Timer Service and Enterprise Scheduling

The EJB Timer Service provides a container-managed, transactional, and robust scheduling mechanism for enterprise applications. In BCD module scenarios, timers are frequently used for batch processing, automated report generation, and periodic system maintenance.

2.1 Declarative Timers (@Schedule)

Declarative timers are created at deployment time based on annotations. The @Schedule annotation allows developers to define a cron-like schedule directly on a business method. This is the simplest way to configure enterprise scheduling.

```
@Stateless
public class ReportGeneratorBean {

    // Runs every Monday at 8:00 AM
    @Schedule(dayOfWeek = "Mon", hour = "8", minute = "0",
persistent = true)
    public void generateWeeklyReport() {
        System.out.println("Generating weekly report...");
        // Report generation logic
    }

    // Runs every 15 minutes
    @Schedule(minute = "*/15", hour = "*", persistent = false)
    public void refreshCache() {
        System.out.println("Refreshing system cache...");
    }
}
```

2.2 Programmatic Timers

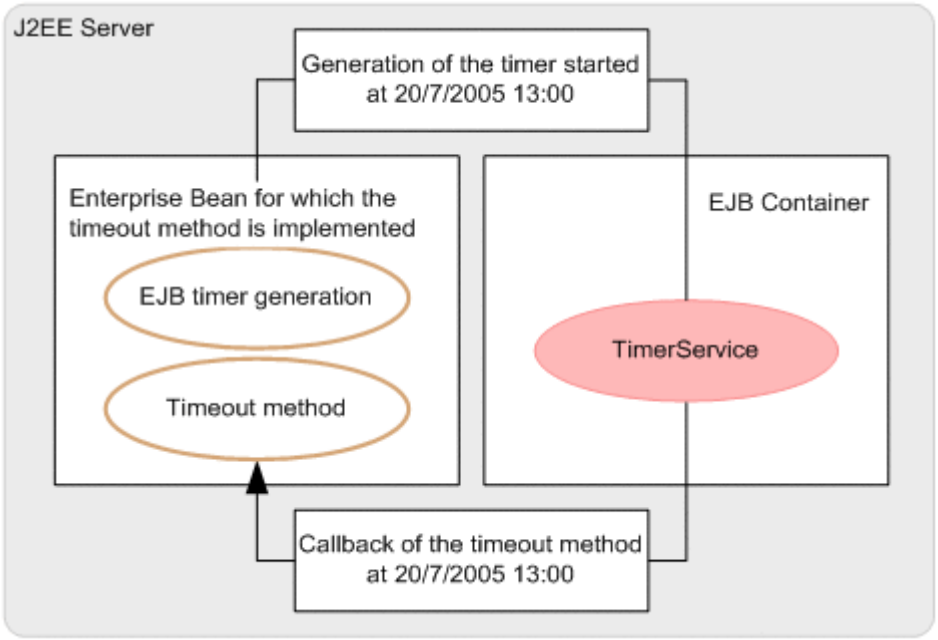
For dynamic scheduling where the interval or target time is determined at runtime (e.g., a user schedules a reminder from a web interface), the programmatic TimerService API must be used.

```
@Stateless
public class NotificationBean {

    @Resource
    private TimerService timerService;

    public void scheduleNotification(Date timeToNotify, String
message) {
```

```
            TimerConfig config = new TimerConfig();
            config.setInfo(message);
            config.setPersistent(true);
            timerService.createSingleActionTimer(timeToNotify,
config);
        }

        @Timeout
        public void executeNotification(Timer timer) {
            String message = (String) timer.getInfo();
            System.out.println("Sending Notification: " + message);
        }
}
```

Diagram: EJB Timer Service Architecture

Description: A component diagram showing the EJB Container, the internal Timer Service engine, the underlying database (for persistent timers), and the target Stateless/Singleton EJB. Arrows indicate the trigger mechanism calling the @Timeout annotated method.



[EJB Timer Service Architecture]

2.3 Persistent vs. Non-Persistent Timers

By default, EJB timers are persistent. This means the container saves the timer's state to a database. If the server crashes or is restarted, the timer survives and will execute when the server comes back online (triggering any missed executions if configured to do so). Non-persistent timers exist only in memory and are lost upon server shutdown. They are ideal for tasks like cache clearing where a missed execution during downtime is irrelevant.

3. Web Services (JAX-WS & JAX-RS)

Web Services enable interoperability between diverse applications over a network. Java EE provides robust support for exposing EJB business components as both SOAP (Simple Object Access Protocol) and REST (Representational State Transfer) web services.

3.1 JAX-WS (SOAP Web Services)

JAX-WS is the API for XML-based web services. By simply annotating a Stateless Session Bean with @WebService, the container automatically generates the WSDL (Web Services Description Language) and handles the SOAP envelope parsing.

```
@Stateless
@WebService(name = "InventoryPortType", serviceName =
"InventoryService")
public class InventoryServiceBean {

    @WebMethod(operationName = "checkStock")
    public int checkStockLevel(@WebParam(name = "productId")
String productId) {
        // Database lookup logic
        return 150;
    }
}
```

3.2 JAX-RS (RESTful Web Services)

RESTful services rely on standard HTTP methods (GET, POST, PUT, DELETE) and generally use JSON or XML for payloads. JAX-RS makes it easy to map HTTP requests to EJB methods using intuitive annotations.

```
@Stateless
@Path("/customers")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class CustomerResource {

    @PersistenceContext
    private EntityManager em;

    @GET
    @Path("/{id}")
    public Customer getCustomer(@PathParam("id") Long id) {
        return em.find(Customer.class, id);
    }

    @POST
    public Response createCustomer(Customer customer) {
        em.persist(customer);
        return
Response.status(Response.Status.CREATED).entity(customer).build()
;
    }
}
```

Diagram: SOAP vs REST Data Flow

Description: A split diagram. The top half shows a SOAP envelope (XML) being wrapped and unwrapped between a Client and an EJB Endpoint. The bottom half shows a lightweight REST architecture where raw JSON over HTTP GET/POST directly interacts with the JAX-RS annotated EJB.

#	SOAP	REST
1	A XML-based message protocol	An architectural style protocol
2	Uses WSDL for communication between consumer and provider	Uses XML or JSON to send and receive data
3	Invokes services by calling RPC method	Simply calls services via URL path
4	Does not return human readable result	Result is readable which is just plain XML or JSON
5	Transfer is over HTTP. Also uses other protocols such as SMTP, FTP, etc.	Transfer is over HTTP only
6	JavaScript can call SOAP, but it is difficult to implement	Easy to call from JavaScript
7	Performance is not great compared to REST	Performance is much better compared to SOAP - less CPU intensive, leaner code etc.

[SOAP vs REST Data Flow]

3.3 Best Practices

• Use JAX-RS (REST) for modern web and mobile applications due to its lightweight nature and native JSON support.

• Use JAX-WS (SOAP) for legacy enterprise integrations where strict contracts (WSDL) and WS-Security are mandatory.

• Always validate inputs at the web service boundary before passing data into core business logic.

4. Interceptors and AOP in Java EE

Interceptors are a mechanism for Aspect-Oriented Programming (AOP) in Java EE. They allow developers to separate cross-cutting concerns—such as logging, auditing, and performance monitoring—from the core business logic of an EJB.

4.1 The @AroundInvoke Annotation

The primary interceptor mechanism revolves around the @AroundInvoke annotation. A method annotated with @AroundInvoke wraps the target business method. It can inspect the method parameters, modify them, decide whether to proceed with the invocation, and manipulate the return value.

```
public class AuditingInterceptor {

    @AroundInvoke
    public Object auditMethod(InvocationContext ic) throws
Exception {
        System.out.println("Entering method: " +
ic.getMethod().getName());
        long startTime = System.currentTimeMillis();

        try {
            // Proceed to the actual EJB method
            Object result = ic.proceed();
            return result;
        } finally {
            long totalTime = System.currentTimeMillis() -
startTime;
            System.out.println("Exiting method: " +
ic.getMethod().getName()
                                + " Execution time: " + totalTime
+ "ms");
        }
    }
}
```

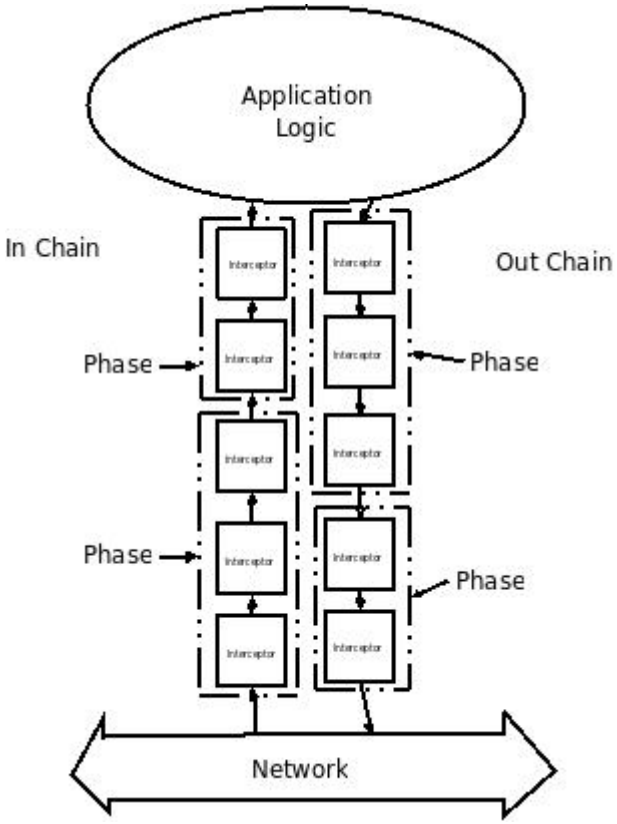
4.2 Applying Interceptors

Interceptors can be applied at the class level, method level, or globally using deployment descriptors.

```
@Stateless
@Interceptors({AuditingInterceptor.class})
public class OrderProcessorBean {

    public void processOrder(Order order) {
        // Business logic here
        // The interceptor runs before and after this block
automatically
    }

    @ExcludeClassInterceptors
    public void healthCheck() {
        // Interceptor will NOT run for this method
    }
}
```

Diagram: Interceptor Chain Execution

Description: A sequence diagram illustrating a Client request passing through multiple Interceptors (e.g., Security, Logging, Validation) before reaching the Target EJB Method, and tracing the return path back through the same interceptors.



[Interceptor Chain Execution]

5. Java EE Security Architecture

Security in enterprise applications involves two main phases: Authentication (verifying who the user is) and Authorization (verifying what the user is allowed to do). EJB security primarily focuses on Authorization.

5.1 Declarative Security

Java EE provides a robust declarative security model based on roles. Developers define logical roles and map them to EJB methods using annotations.

```
@Stateless
@DeclareRoles({"ADMIN", "USER", "MANAGER"})
public class EmployeeServiceBean {

    @RolesAllowed({"ADMIN", "MANAGER"})
    public void promoteEmployee(Long empId) {
        // Logic accessible only to ADMIN or MANAGER
    }

    @PermitAll
    public Employee getEmployeeDetails(Long empId) {
        // Accessible by anyone, including unauthenticated users
    }

    @DenyAll
    public void purgeSystem() {
        // Nobody can call this from the outside
    }
}
```

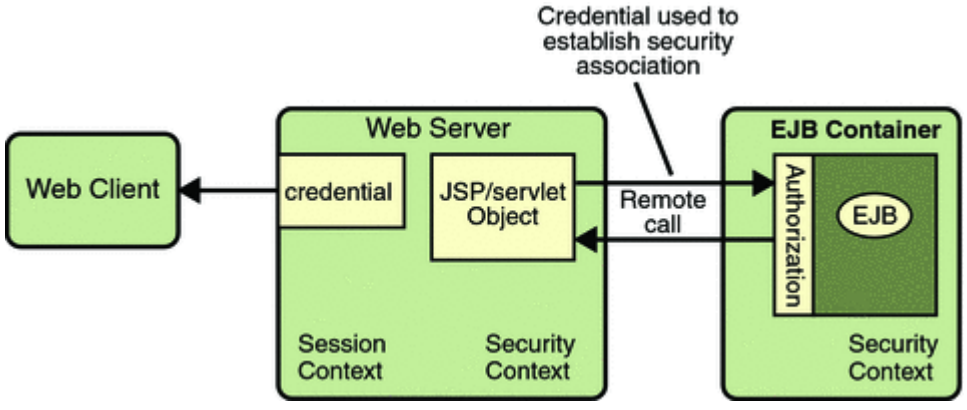
5.2 Programmatic Security

When role-based access control is not granular enough (e.g., a user can edit an entity, but only if they are the creator of that entity), programmatic security is used via the SessionContext.

```
@Stateless
public class DocumentManagerBean {

    @Resource
    private SessionContext ctx;

    public void deleteDocument(Document doc) {
        Principal caller = ctx.getCallerPrincipal();

        if (ctx.isCallerInRole("ADMIN") ||
doc.getOwner().equals(caller.getName())) {
            // Proceed with deletion
        } else {
            throw new EJBAccessException("Unauthorized access to
document deletion");
        }
    }
}
```

Diagram: EJB Security Context Evaluation

Description: A flowchart showing how the EJB container intercepts a remote call, evaluates the Principal, checks the Security Realm for mapped roles, and compares it against the @RolesAllowed annotations on the EJB method.



[EJB Security Context Evaluation]

6. EJB Exception Handling Strategies

Exception handling in EJB is unique because the container intercepts exceptions to decide the fate of the transaction and the bean instance. Exceptions are categorized into Application Exceptions and System Exceptions.

6.1 Application Exceptions vs System Exceptions

• Application Exceptions: These are expected business errors (e.g., InsufficientFundsException). They are typically checked exceptions (extending java.lang.Exception). When thrown, the container does NOT automatically roll back the transaction (unless specified) and does NOT destroy the bean instance.

• System Exceptions: These are unexpected technical errors (e.g., NullPointerException, SQLException). They are unchecked exceptions (extending java.lang.RuntimeException). When thrown, the container logs the error, ROLLS BACK the current transaction, and DESTROYS the bean instance to prevent state corruption.

6.2 The @ApplicationException Annotation

You can customize exception behavior using the @ApplicationException annotation, particularly to force a transaction rollback for a business exception.

```
@ApplicationException(rollback = true)
public class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

@Stateless
public class BankingServiceBean {

    public void transferFunds(Account from, Account to, double
amount)
           throws InsufficientFundsException {

        if (from.getBalance() < amount) {
            // Container will catch this and roll back the
transaction
            throw new InsufficientFundsException("Not enough
money.");
```

```
            }
            // Transfer logic
        }
    }
```

7. Packaging and Deploying EJB Applications

Java EE applications are packaged into standardized archive formats. Understanding these structures is critical for successful deployment to application servers like WildFly, GlassFish, or WebLogic.

7.1 Archive Types

• JAR (Java Archive): Used to package EJB components. Contains the compiled .class files and an optional META-INF/ejb-jar.xml deployment descriptor.

• WAR (Web Archive): Used for web components (Servlets, JSP, HTML, REST endpoints). Placed in WEB-INF/classes or WEB-INF/lib.

• EAR (Enterprise Archive): The master wrapper that contains multiple JARs and WARs, representing a complete enterprise application. Configured via META-INF/application.xml.

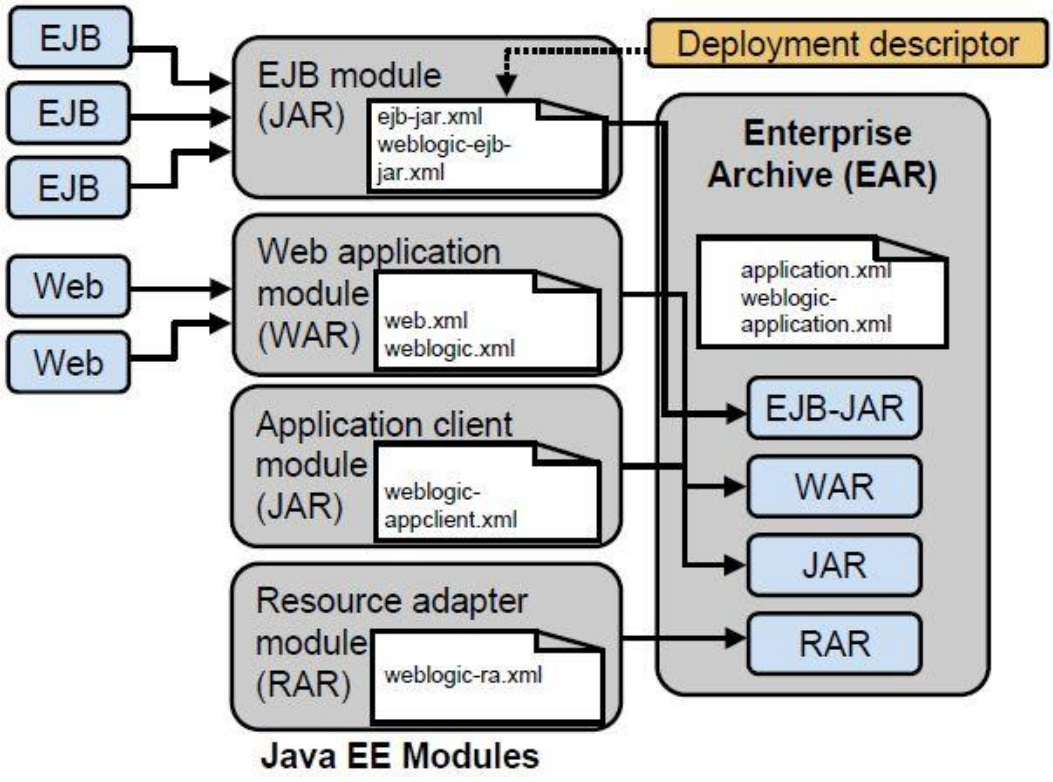
7.2 The ejb-jar.xml Descriptor

While modern Java EE heavily relies on annotations, deployment descriptors can override annotation metadata at deployment time without recompiling the code. This is useful for environment-specific configurations.

```
<ejb-jar xmlns="http://xmlns.jcp.org/xml/ns/javaee"
version="3.2">
    <enterprise-beans>
        <session>
            <ejb-name>OrderServiceBean</ejb-name>
            <transaction-type>Container</transaction-type>
        </session>
    </enterprise-beans>
    <assembly-descriptor>
        <container-transaction>
            <method>
                <ejb-name>OrderServiceBean</ejb-name>
                <method-name>*</method-name>
            </method>
            <trans-attribute>RequiresNew</trans-attribute>
        </container-transaction>
    </assembly-descriptor>
</ejb-jar>
```

Diagram: EAR Archive Hierarchy

Description: A directory tree diagram showing the structure of an EAR file. Root contains META-INF/application.xml. Below root are ejb-module.jar (containing stateless beans and META-INF/ejb-jar.xml) and web-module.war (containing JSF pages and WEB-INF/web.xml).



[EAR Archive Hierarchy]

7.3 Classloading in Java EE

Classloading in an EAR file typically follows a hierarchical model. The application server classloader loads container libraries. The EAR classloader loads shared libraries in the EAR's lib directory. Sub-classloaders are created for the EJB module and Web module. Understanding this isolation prevents ClassNotFoundExceptions and ClassCastExceptions.